

## Application Based Questions

← Back

## QUESTIONS

Q1

Student Grade Book  
Application  
10 marks

Q2

Debugging and Rewriting  
10 marks

2/2 answered

## Q1 Student Grade Book Application

10 marks

You are building a student grade book application. Write a Python program that:

- (a) Stores each student's name and marks in a dictionary (key = name, value = list of 5 subject marks).
- (b) Computes total, average, and grade (D:  $\geq 90$ , A+:  $\geq 80$ , A:  $\geq 70$ , B+:  $\geq 60$ , B:  $\geq 50$ , F:  $< 50$ ) for each student.
- (c) Finds the class topper (highest average) and the subject with the highest overall mean.
- (d) Displays a formatted result table sorted by average in descending order.

Use dictionary comprehension and list operations. Minimum 5 students.  
Write complete, syntactically correct Python programs using lists, tuples, dictionaries, and sets with proper indentation.

## Your Code

```
1 students = {
2     "Arun": [85, 90, 78, 92, 88],
3     "Bala": [70, 65, 80, 75, 60],
4     "Chitra": [95, 92, 88, 91, 96],
5     "Divya": [55, 60, 48, 52, 58],
6     "Elango": [40, 45, 38, 50, 42]
7 }
8
9 def grade(avg):
10     for limit, g in [(90, "D"), (80, "A+"), (70, "A"), (60, "B+"), (50, "B")]:
11         if avg >= limit:
12             return g
13     return "F"
14
15 results = [(n, (sum(m), sum(m)/5, grade(sum(m)/5)) for n, m in students.items())
16
17 topper = max(results, key=lambda n: results[n][1])
18 sub_means = [sum(students[n][i] for n in students) / len(students) for i in range(5)]
19 best_subject = sub_means.index(max(sub_means)) + 1
20
21 print(f"{'Name':<10}{'Total':<5}{'Average':<10}{'Grade':<6}")
22 for name in sorted(results, key=lambda n: results[n][1], reverse=True):
23     t, s, g = results[name]
24     print(f"{name:<10}{t:<5}{s:<10.2f}{g:<6}")
25
26 print(f"\nTopper: {topper} (Avg = {results[topper][1]:.2f})")
27 print(f"Best Subject: Subject{best_subject} (Mean = {max(sub_means):.2f})")
```

## Input

stdin input

## Output

| Name   | Total | Average | Grade |
|--------|-------|---------|-------|
| Chitra | 463   | 92.60   | D     |
| Arun   | 433   | 86.60   | A+    |
| Bala   | 350   | 70.00   | A     |
| Divya  | 273   | 54.60   | B     |

```
create_account(1001, 'Ravi', 10000)
deposit(1001, 5000)
withdraw(1001, 20000)
withdraw(1001, 3000)
mini_statement(1001)
print('Balance:', accounts[1001]['balance'])
```

For debugging questions: clearly state the error, the line number, the reason, and the corrected line.

#### Your Code

```
1 accounts = {}
2
3 def create_account(acc_no, name, balance):
4     accounts[acc_no] = {'name': name, 'balance': balance, 'txns': []}
5
6 def deposit(acc_no, amount):
7     if acc_no in accounts:
8         accounts[acc_no]['balance'] += amount
9         accounts[acc_no]['txns'].append(('Deposit', amount))
10
11 def withdraw(acc_no, amount):
12     if acc_no in accounts and accounts[acc_no]['balance'] >= amount:
13         accounts[acc_no]['balance'] -= amount
14         accounts[acc_no]['txns'].append(('Withdraw', amount))
15     else:
16         print('Insufficient balance')
17
18 def mini_statement(acc_no):
19     for t in accounts[acc_no]['txns'][-5:]:
20         print(t)
21
22 create_account(1001, 'Ravi', 10000)
23 deposit(1001, 5000)
24 withdraw(1001, 20000)
25 withdraw(1001, 3000)
26 mini_statement(1001)
27 print('Balance:', accounts[1001]['balance'])
```

 Run

 Save

#### Input

stdin input

#### Output

```
Insufficient balance
('Deposit', 5000)
('Withdraw', 3000)
Balance: 12000
```